



Data Science Internship Program • Summer 2026

Lahore Real Estate Price Prediction Pipeline

Intern Assignment Specification

Week 2 — Advanced Predictive Modeling & Optimization

"From a working predictor to a defensible, production-ready property intelligence engine."

DOMAIN	LEVEL	DURATION	STACK
Real Estate / Data Science	Intermediate–Advanced	7 Days	Python · scikit-learn · XGBoost · SHAP

1. Overview

This assignment builds on a genuine property price prediction pipeline for the Lahore residential real estate market — one that engineers structured features from raw listing data (area, location, property type, floor, amenities), applies target encoding for high-cardinality categorical variables, enforces a strict random-stratified train/test split, and includes a baseline interpretability layer using SHAP. The starting pipeline should already be validated as leak-free, delivering point-estimate predictions alongside feature importances.

Week 2 pushes that pipeline from "working" to "defensible": you will tune it rigorously against a proper validation scheme, benchmark it honestly against at least one competing model family, prune and justify your feature set with evidence, quantify uncertainty in every price prediction, and prepare the winning model for actual deployment — measuring latency, artifact size, and exposing a clean load-and-predict interface. The bar this week is not just a lower RMSE; it is proving, with logged evidence, that your model is well-tuned, well-validated, and genuinely production-viable.

Why This Matters — Real Estate Context

- Lahore's residential market spans a wide price range — from budget apartments in Johar Town to multi-crore plots in DHA Phase 6 — making a robust, uncertainty-aware predictor genuinely valuable.
- Buyers and agents need not just a price estimate but a confidence interval: 'this 5-marla house in Bahria Town should sell for PKR 2.1–2.6 Cr at 90% confidence.'
- A model that fails silently on unusual listings (basement flats, corner plots, under-construction) is worse than no model — this week you will find and quantify those failure zones.

2. Technology Stack (Python / Data Science)

Continue on your existing modular codebase — no notebook dumps as final deliverables. All new modules must be clean, type-hinted Python files. The stack below is the minimum required; you may add libraries only if justified in the write-up.

Component	Details
Language	Python 3.10+ — clean, type-hinted, modular (no top-level notebook cells in final pipeline)
Data Layer	Pandas, NumPy — existing feature pipeline extended, not replaced
Optimization	scikit-learn GridSearchCV / RandomizedSearchCV with StratifiedKFold or KFold; or Optuna / Hyperopt for Bayesian search on models with large hyperparameter spaces
Core Models	Baseline model (e.g. XGBoost / LightGBM / Random Forest) + at least one additional family (e.g. Ridge/Lasso regularized regression, CatBoost, or a stacked ensemble)
Interpretability	SHAP — updated for tuned model + permutation importance for feature elimination pass
Uncertainty	Quantile regression (XGBoost quantile objective / sklearn QuantileRegressor) or bootstrapped prediction intervals or MAPIE conformal prediction wrapper
Serialization	joblib (preferred for sklearn/XGBoost), or native save_model() for LightGBM/XGBoost; include a documented load-and-predict interface
Logging	Python logging module or MLflow (lightweight) for recording all hyperparameter trial results — no single 'best params' printout without the search log

Component	Details
Optional	Jupyter notebooks for exploratory tuning only — final deliverable pipeline must be importable Python modules

3. Core Requirements

3.1 Hyperparameter Optimization

Your baseline model used default or hand-picked parameters. This week you must systematically search the hyperparameter space with a validation strategy appropriate for structured property data:

- Use KFold or StratifiedKFold cross-validation (5 folds minimum). Stratify on a binned price range to ensure each fold has similar price distribution.
- Search strategy — use GridSearchCV for small parameter grids, RandomizedSearchCV for medium grids, or Optuna/Hyperopt for large spaces (XGBoost: max_depth, learning_rate, n_estimators, subsample, colsample_bytree, min_child_weight, reg_alpha, reg_lambda).
- Log every trial — parameters + validation RMSE (in PKR and as a percentage of mean price). The log must be saved as a CSV or JSON file.
- Report the best configuration found and what the search revealed about hyperparameter sensitivity: e.g. 'validation RMSE plateaued past n_estimators=400, and max_depth beyond 6 caused overfitting.'
- Avoid log-scale target leakage — if you apply $\log_{10}()$ to price, ensure all CV scoring is done in original PKR via inverse-transform before computing RMSE/MAPE.

Lahore-Specific Tuning Tip

- DHA and Bahria Town listings often form separate price clusters. If RMSE is suspiciously low, check whether the model is exploiting location encoding as a near-perfect price proxy.
- Prices range from ~20 lakh to 10+ crore. Always report RMSE in raw PKR and as a percentage of mean/median price — raw PKR RMSE alone is uninterpretable.

3.2 Ensemble & Comparative Modeling

A single tuned model is not enough this week. You must demonstrate that your model choice is actually the best one on this dataset:

- Train at least one additional model family distinct from your baseline choice. Suggested pairs:
 - Baseline = XGBoost → add Ridge/Lasso regularized regression as a strong linear baseline
 - Baseline = Random Forest → add LightGBM or CatBoost
 - Baseline = Linear model → add a gradient-boosted tree (XGBoost/LightGBM)
- Optionally build a simple ensemble/blend — a weighted average or stacking ensemble with a linear meta-learner. Test whether it beats the best single model and report the delta.
- All candidates evaluated identically — same held-out validation split, same preprocessed features, identical metrics. No cherry-picking.
- Justify your final model choice using the comparison table — accuracy, training time, inference latency, interpretability, and maintenance burden all count.

Model Comparison Table Format (minimum):

Model	RMSE (PKR)	RMSE (%)	MAPE (%)	MAE (PKR)	Train Time	Latency/ pred	Final ?
Baseline Model (untuned)	—	—	—	—	—	—	No

Model	RMSE (PKR)	RMSE (%)	MAPE (%)	MAE (PKR)	Train Time	Latency/ pred	Final ?
Baseline Model (tuned)	—	—	—	—	—	—	TBD
Additional Model Family	—	—	—	—	—	—	TBD
Ensemble / Blend (if built)	—	—	—	—	—	—	TBD

3.3 Feature Selection & Dimensionality Management

Your baseline feature set was broad — property type dummies, location target encoding, area, floor, bedroom/bathroom counts, age, binary amenity flags. This week, prove which features are actually earning their place:

- Run a systematic feature importance pass using SHAP values AND permutation importance (to cross-validate — SHAP can overstate importance of correlated features).
- Identify and justify removal of redundant or low-signal features. Show the pairwise correlation heatmap for highly correlated amenity flags.
- Re-validate trimmed feature set — confirm RMSE/MAPE does not worsen meaningfully (more than 2–3%) versus the full set. Show the before/after table.
- If you engineer new features this week, justify each with validation RMSE evidence. Suggested candidates for Lahore real estate:
 - price_per_sqft derived feature (for detecting outlier listings)
 - society_tier: DHA/Bahria = premium, Johar/Gulberg = mid, other = standard (ordinal)
 - bedroom_bath_ratio: bedrooms / bathrooms — proxy for apartment vs. house layout
 - log1p(area_sqft) — reduces right-skew in the area distribution
- No raw integer encoding for categories — location, property type, and society must remain target-encoded or one-hot encoded where cardinality is low.

3.4 Robustness, Uncertainty & Error Analysis

Point price estimates alone are not sufficient. A buyer considering a PKR 3.5 Cr house needs to know whether your model is confident within ± 5 lakh or uncertain within ± 50 lakh:

- Produce prediction intervals alongside every point forecast using one of:
 - Quantile regression — XGBoost with quantile:0.05 and quantile:0.95 objectives for 10th/90th percentile estimates
 - Bootstrapped residual intervals — spread of predictions across N bootstrap samples on each validation listing
 - Conformal prediction via MAPIE — wraps any sklearn-compatible model with coverage-guaranteed intervals
- Validate empirical coverage — on the held-out set, verify the 90% PI actually contains the true price ~90% of the time. Report the actual coverage rate.
- Residual analysis — plot residuals against: (a) predicted price level, (b) property area, (c) location cluster. Identify systematic patterns.
- Explicit stress-scenario tests — test on at least two of:
 - Listings with unusual area-to-bedroom ratios (e.g. 10-marla 1-bed)
 - Very high-value listings (top 1% by price) — does RMSE spike disproportionately?
 - Out-of-distribution locations with fewer than 5 training examples
 - Missing-amenity listings (all amenity flags = 0)
- Leakage re-audit — confirm no new feature added this week is computable only from the target price. Document each new feature's source.

3.5 Model Optimization for Deployment

Take the winning model from Section 3.2 and treat it as something that must actually serve predictions in a production environment — a backend API that real estate agents might query in real time:

- Measure and report inference latency per prediction and per batch (100 listings). Use `time.perf_counter()` or `timeit`. Report median latency across 1000 runs.
- Measure artifact size on disk — serialize the full pipeline with `joblib` and report the `.pkl/.joblib` file size in MB.
- Apply at least one optimization step and report the trade-off explicitly:
 - Reduce ensemble estimators — e.g. XGBoost 500 trees → 200 trees; report RMSE delta vs. latency/size gain
 - Drop bottom N% features by SHAP importance → retrain, measure latency and RMSE impact
- Serialize the final pipeline using `joblib.dump()` on the complete sklearn Pipeline object (preprocessor + model in one object). Provide a documented `predict()` wrapper:

```
# deployment/load_and_predict.py – documented interface
def load_model(path: str) -> Pipeline: ...
def predict_price(model, listing: dict) -> dict:
    # returns {price_pkr, lower_90, upper_90}
```

3.6 Code Quality & Project Structure

Extend — do not replace — your existing project structure:

Component	Details
tuning/	Hyperparameter search scripts, logged trial CSV/JSON, best-config output
models/	Updated train/predict wrappers, new comparison model(s), ensemble logic, serialized <code>.joblib</code> artifact
evaluation/	Extended metrics, residual analysis plots, uncertainty/coverage validation, model comparison table
features/	Updated feature engineering pipeline with feature-selection logic and importance rankings
deployment/	<code>load_and_predict.py</code> (documented interface), latency benchmarking script, optimization report
notebooks/	Exploration and EDA only — NOT the final deliverable pipeline
data/	Raw and processed data (or symlinks/instructions to download Zameen.com dataset)
README.md	Updated with tuning strategy, model comparison rationale, feature-selection summary, deployment profile

AI Usage Policy: You may use AI tools to research library APIs or debug code. However, the search strategy, model comparison logic, feature-selection rationale, and final model justification must be understood and authored by you.

4. Data Source & Lahore Real Estate Domain Guide

Understanding the data domain is as important as model performance. This section describes the expected dataset structure, known data quality issues, and domain knowledge that should inform your feature engineering and error analysis.

4.1 Primary Dataset — Zameen.com Listings

Your dataset is a scraped or downloaded export of Lahore residential property listings from Zameen.com (or an equivalent local source). The typical schema:

Column	Type	Description	Notes / Issues
price	int/float	Listing price in PKR	Heavy right-skew; outliers present
location	string	Sub-area (e.g. Phase 6 DHA)	High cardinality — use target encoding
city_area	string	Broad area (DHA, Bahria, Gulberg...)	Use as society_tier feature
property_type	category	House / Flat / Upper Portion / Plot	Low cardinality — one-hot OK
area_sqft	float	Total area in sq ft	Sometimes in marla — normalise
area_marla	float	Total area in marla	1 marla = 225 sqft; derive one from other
bedrooms	int	No. of bedrooms	Missing values common; impute median by type
bathrooms	int	No. of bathrooms	Correlated with bedrooms
purpose	category	For Sale / For Rent	Filter to For Sale only
furnishing	category	Furnished / Semi / Unfurnished	May be missing; treat as category
amenities	multi-label	Comma-separated list	Parse into binary flag columns
date_added	datetime	Listing date	Useful for recency weighting if needed

4.2 Known Data Quality Issues

- Duplicate listings — the same property relisted multiple times. Deduplicate on (location, area_sqft, bedrooms, bathrooms) with a price tolerance of $\pm 2\%$.
- Price outliers — listings at PKR 1 (placeholder) or PKR 10 billion (data entry errors). Filter to PKR 10 lakh – PKR 50 crore for residential properties.
- Area unit inconsistency — marla, sq ft, and kanal (1 kanal = 20 marla = 4500 sqft). Normalise everything to square feet.
- Missing amenities — treat absence as a distinct category (is_amenities_listed=0) rather than imputing.
- Location granularity mismatch — 'DHA Phase 1', 'Defence Phase 5', 'DHA Phase 6' etc. Normalise using a lookup table to a consistent society-level label.

5. Deliverables

All deliverables must be submitted as a single Git repository link with a clean commit history and a properly updated README. Do not submit notebooks alone.

D1 — Updated Source Code Repository

Building on your existing codebase with all new modules (tuning/, deployment/ etc.). README must include: tuning strategy, models compared and why, feature-selection rationale, deployment latency/size profile, and a 'What's New This Phase' section.

D2 — Results Report

A short PDF or markdown report containing: model comparison table with RMSE/MAPE/MAE for all candidate models, hyperparameter search summary, predicted-vs-actual price scatter plot for the winning model, prediction-interval/coverage validation plot, residual analysis plots with written interpretation, and stress-scenario results table.

D3 — Optimization Trade-off Note

A concise before/after table: artifact size (MB), median single-prediction latency (ms), per-batch latency (100 listings), and validation RMSE — for both unoptimized and optimized versions of the final model.

D4 — Serialized Production Model

A .joblib file containing the full preprocessor + model pipeline, accompanied by deployment/load_and_predict.py — a documented, importable script with predict_price() returning {price_pkr, lower_90, upper_90} given a raw listing dict.

D5 — 'What's New This Phase' Summary

A one-page section in README (or CHANGELOG.md) describing: what was tuned, what new model was added and why, which features were removed/added and why, and how the deployment profile improved from the original unoptimized baseline.

6. Evaluation Criteria

Your submission will be evaluated across six dimensions. Each carries equal weight. A submission that excels in three areas but is missing two entirely will not pass.

Criterion	What We Look For	Common Failure
Tuning Rigor	Genuine logged hyperparameter search with CV. Search log saved as CSV/JSON. Sensitivity analysis written.	Single hand-picked parameter set with no search log.
Honest Model Comparison	2+ model families on the same held-out data. Comparison table with all metrics. Final choice justified beyond RMSE.	Only one model evaluated, or different models on different data slices.
Feature Selection Quality	SHAP + permutation importance run. Redundant features removed with evidence. Before/after RMSE shown.	Features added/removed without evidence; importance scores not shown.
Uncertainty Quantification	Prediction intervals produced. Empirical coverage validated (~90% for 90% PI). Intervals shown on plot.	Only point forecasts delivered; no coverage validation.
Error Analysis Depth	Residuals plotted vs. price, area, location. At least two stress scenarios tested with quantified error.	Only aggregate RMSE reported; no residual patterns explored.
Deployment Readiness	Latency + size measured. One optimization applied with trade-off reported. <code>load_and_predict.py</code> works.	Model serialized but no latency measured; or <code>load_and_predict.py</code> missing.

7. Self-Check Checklist

Complete every item below before submitting. This checklist is used as a marking rubric.

- Hyperparameter search performed with KFold / StratifiedKFold CV (price-binned stratification) — not a single hand-picked configuration.
- All search trials logged (parameters + validation RMSE in PKR and %) to a CSV or JSON file. Search is reproducible.
- At least one additional model family trained and evaluated on the same validation split as the baseline model.
- Model comparison table reports RMSE (PKR + %), MAPE, and MAE on the same held-out validation set for all candidate models.
- Feature importance/elimination pass completed using SHAP AND permutation importance. Redundant features identified and removed.
- Before/after RMSE comparison table shows trimmed feature set does not hurt performance (threshold: < 3% RMSE increase).
- Prediction intervals (90% PI) or quantile estimates produced alongside point price estimates.
- Empirical coverage of prediction intervals validated: 90% PI should contain true price ~90% of the time.
- Residual analysis completed — residuals plotted vs. price level, area, and location cluster.
- At least two stress scenarios tested with error quantified (e.g. top-1% listings, sparse-location listings).
- Leakage re-audit completed — no new feature computable only from the target price was added.
- Final model's median single-prediction latency and artifact size measured and reported.
- At least one deployment optimization applied with explicit before/after trade-off table.
- Final model serialized as a complete pipeline (.joblib) — preprocessor + model in one artifact.
- deployment/load_and_predict.py written, documented, and tested: returns {price_pkr, lower_90, upper_90}.
- Project structure extended (tuning/, deployment/, updated models/ and evaluation/) without breaking existing imports.
- 'What's New This Phase' summary included in README or CHANGELOG.md.

8. 7-Day Execution Timeline

This is a structured timeline — not a suggestion. If you fall behind on Day 2, compress earlier tasks rather than skipping Day 5's uncertainty quantification.

Day 1 — Review & Search Setup

- › Review your existing pipeline: re-run it, check metrics, identify weakest points.
- › Define hyperparameter search space (document in `tuning/search_config.py`).
- › Set up CV harness (StratifiedKFold on price bins) and confirm stable fold distributions.
- › Ensure log1p price transformation is in place with inverse-transform in scoring.

Day 2 — Hyperparameter Search

- › Run full hyperparameter search (GridSearch / RandomizedSearch / Optuna).
- › Save all trial logs to `tuning/trial_log.csv`. Plot RMSE vs. key hyperparameters.
- › Select best configuration; write sensitivity analysis narrative.
- › Confirm tuned model beats the untuned baseline.

Day 3 — Comparative Modeling

- › Train and tune the additional model family on the same training split.
- › Optionally: build and evaluate a simple ensemble/blend.
- › Populate the model comparison table — RMSE/MAPE/MAE/latency for all candidates.
- › Select the winning model with a written justification (not just 'lowest RMSE').

Day 4 — Feature Selection

- › Run SHAP feature importance on the winning model; plot mean |SHAP| per feature.
- › Run permutation importance as a cross-check; compare rankings.
- › Identify bottom quartile features; retrain on trimmed set; record RMSE delta.
- › Validate any new features (`price_per_sqft`, `society_tier`) with hold-out RMSE test.

Day 5 — Uncertainty & Residual Analysis

- › Implement prediction intervals (quantile regression, bootstrap, or conformal prediction).
- › Validate empirical coverage: what % of validation prices fall inside the 90% PI?
- › Plot residuals vs. predicted price, area, and location cluster.
- › Run at least two stress scenarios; document error increases.

Day 6 — Deployment Optimization

- › Measure baseline inference latency (single prediction + batch of 100) and artifact size.
- › Apply at least one optimization (reduce estimators, drop bottom features, etc.).
- › Record before/after trade-off table; confirm RMSE did not degrade beyond tolerance.
- › Write and test deployment/`load_and_predict.py` — run on 10 sample listings.

Day 7 — Report, README & Clean-up

- › Write up results report (PDF or markdown) with all required plots and tables.

- › Update README: tuning strategy, model comparison rationale, feature-selection summary, deployment profile, 'What's New This Phase' section.
- › Run the self-check checklist (Section 7) — ensure every box can be ticked.
- › Final commit; tag as 'week2-submission'; submit repository link.

9. Lahore Real Estate Domain Reference

Domain context helps you make better modelling and feature engineering decisions. A data scientist who understands the domain makes better decisions at every step.

9.1 Key Property Categories in Lahore

Category	Examples	Typical Price Range (PKR)	Modelling Notes
Premium Plots	DHA Phase 5–8, Bahria Orchard	1.5 Cr – 10+ Cr	High variance; may need separate cluster flag
Premium Houses	DHA, Bahria Town Phase 4–8	2 Cr – 15+ Cr	Corner/park-facing premiums significant
Mid-Range Houses	Johar Town, Gulberg, Model Town	80 L – 3 Cr	Most common listings; model should perform best here
Budget Apartments	Johar Town, Allama Iqbal Town	25 L – 1.2 Cr	Floor level, elevator access matter more
Upper Portions	Various older localities	20 L – 80 L	Often listed without full amenity data
Commercial (exclude)	Main Boulevard, Gulberg	N/A	Filter out; commercial price drivers very different

9.2 High-Impact Features in Lahore Market

Feature	Signal Strength	Implementation
Location (sub-society)	Very High	Target encoding on training split mean price — most important single feature
Area (sqft / marla)	Very High	Normalise to sqft; apply log1p to reduce skew; model non-linearly
Property Type	High	House vs. Flat vs. Plot are fundamentally different — consider type-specific models
Society Tier	High	DHA/Bahria premium vs. mid vs. budget ordinal tier — engineered from location
Bedrooms × Bathrooms	Medium	Strong for apartments; use interaction term
Furnishing Status	Medium	Furnished adds 10–25% premium for apartments
Corner / Park Facing	Medium–High	10–20% premium for premium plots; often missing in data
Age of Property	Medium	Newer properties command premium
Amenities (count)	Low–Medium	Aggregate flag count often more useful than individual flags
Floor Number	Low–Medium	Significant for high-rises; less for 2-storey houses

10. Grading Breakdown

Your Week 2 submission is graded out of 15 marks distributed across four evaluation components. Partial credit is awarded for incomplete but genuine attempts. Zero credit is given for deliverables that are missing, placeholder text, or inaccessible.

Component	Marks	What Is Evaluated	Zero-Credit Condition
GitHub Repository	3 / 15	Clean, well-structured repo with meaningful commit history. Updated README. All required modules present (tuning/, models/, evaluation/, deployment/). Working load_and_predict.py. Code must be publicly accessible at submission time.	Repo missing, private/inaccessible, or contains only notebooks with no module structure.
Code Quality	4 / 15	Type-hinted, modular Python code extending the existing structure. Hyperparameter search script with logged trials (CSV/JSON). Comparison model training script. Feature-selection logic. Serialized pipeline artifact. Latency benchmark script. All scripts runnable end-to-end.	Code is a single unstructured notebook, or core scripts are missing / throw errors on run.
LinkedIn Post	3 / 15	A professional post published on LinkedIn summarising the project: problem statement, approach taken, key result (RMSE in PKR, coverage of prediction intervals), at least one visual (plot or screenshot of results), Khizex tag, and relevant hashtags (#DataScience #MachineLearning #RealEstate #Lahore). Submit the post URL with your repo.	Post not published before submission deadline, contains no technical content, or has no visual/result included.
Interview	5 / 15	Live Q&A; session covering: (1) why you chose this hyperparameter search strategy and what the search revealed, (2) how you validated prediction interval coverage and what the result was, (3) what patterns the residual analysis revealed, (4) which features were dropped and the evidence behind that decision, (5) what the deployment optimization trade-off looked like in numbers. Answers must reflect genuine understanding of your own code.	Unable to explain any modelling decision made in the submission, or answers are clearly memorised without understanding.
Total	15 / 15	—	—

Marks at a Glance

- GitHub Repository · 3 marks — Accessible, structured repo with commit history and complete module layout.
- Code Quality · 4 marks — All scripts runnable, type-hinted, modular; search log present; pipeline serialized.
- LinkedIn Post · 3 marks — Published post with problem, results, visual, Khizex tag, and hashtags.
- Interview · 5 marks — Live Q&A; you must be able to explain every modelling decision in your own words.

Khizex Data Science Internship — Summer 2026 | Week 2 Assignment Specification | Lahore Real Estate Price Prediction Pipeline